Detecting Synthetic Images in AWS SageMaker

In this project I attempted to use the CIFAKE: Real and AI-Generated Synthetic Images dataset from kaggle for Computer Vision classification.

This project serves as the capstone project for the AWS Machine Learning Nanodegree by Udacity. In this project I leveraged the usage of AWS SageMaker for preprocessing the data, tuning hyperparameters, training a model, and deploying the model as an endpoint for inference. I also provide a Step Functions workflow that allows the triggering of a Lambda Function to predict whether an image uploaded to an S3 bucket is real or fake.

This project is motivated by the raise of Generative AI, and the rapid improvement of its capabilities in image generation. This presents huge opportunities and technological advancements, however, worries about the ability of automatic systems (or even people) to differentiate real from fake images have also been growing.

Project Setup

The project consists of the following files:

- `train_and_deploy.ipynb`: this notebook contains all the step for dataset analysis, hyperparameter tuning, training of the final model, and deployment of the endpoint.
- `hpo.py`: this Python code contains the creation, training and testing of a fined-tuned Inception v3 neural network, using hyperparameters given from the arguments.
- `train_model.py`: this Python code contains the creation, training and testing of a fined-tuned Inception v3 neural network, using hyperparameters given from the arguments, and adding a hook for the Sagemaker Profiler and Debugger.
- `requirements.txt`: this file contains the requirements for the hpo and train_model Python files. This is required since the notebook launches the training jobs in a new machine which doesn't have all requirements installed. This file tells Sagemaker which Python libraries to install.
- `README.md`: a comprehensive documentation of the project.

To run this project completely, we only have to execute all the steps in the `train_and_deploy` notebook.

Libraries/Frameworks used

- Black formatter: PEP8 formatter that helps the project code keep on track with standards.
- Kaggle: Website that hosts the dataset used for this model.
- Python3.10: Latest version of the Python language available in Sagemaker Notebook kernels.

Development of the project

This project was developed using the resources offered during the AWS Machine Learning Engineer Nanodegree. This includes an AWS account with access to create resources like SageMaker instances, Lambda functions, etc.

Sagemaker preparation

Firstly I created a Notebook Instance in AWS SageMaker called `detecting-synthetic-images-sagemaker`. This allowed me to access a Jupyter environment with Sagemaker and other libraries installed. The instance type for the notebook chosen is the `ml.t2.medium`. I chose this option because no pre-processing of the data is required, and this is the cheapest option available. This instance costs $0.0464 per hour, and was shut down when the notebook was not in use.

I also created the S3 bucket where the dataset will reside: `detecting-synthetic-images-sagemaker`. I created an Execution Role with full access to this S3 bucket: `AmazonSageMaker-ExecutionRole-20260311T094833`. This is the role the Sagemaker notebook instance used.

Finally, I cloned the present repository inside the Notebook Instance.

Dataset exploration and preprocessing

The dataset used is CIFAKE: Real and AI-Generated Synthetic Images available in Kaggle.

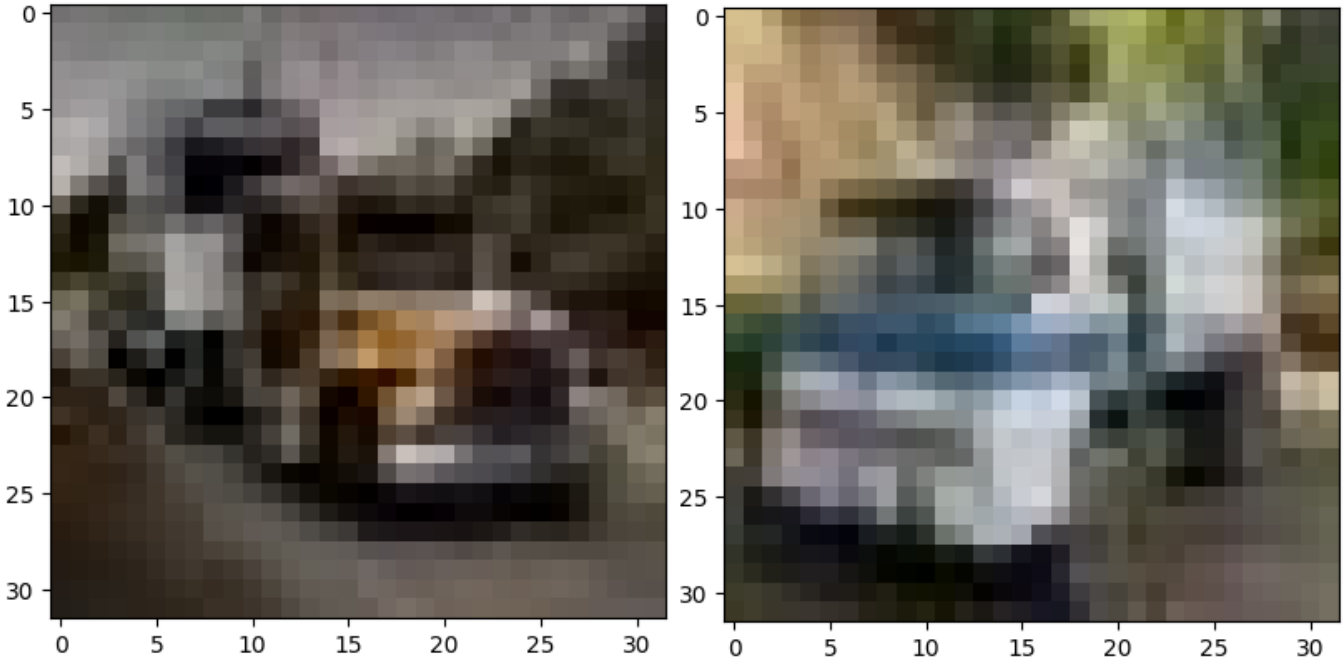
To download and upload the data to S3, I used the Notebook instance created previously. The `train_and_deploy.ipynb` notebook contains cells for downloading the data in a .zip file using the `wget` command, unzipping it in the local Jupyter Folder, and uploading it to S3 into the URI `s3://detecting-synthetic-images-sagemaker/input_data/cifake`.

The dataset is already split in two folders: `train` and `test`. Each of these folders contains two folders, one labeled 'real' and one 'fake'. These are our class labels.

The data is divided as follows:

Dataset	# of images	% of images labeled REAL	% of images labeled FAKE
train	100,000	50%	50%
test	20,000	50%	50%

Every image is 32x32 pixels, with three channels (RGB). Here are two examples of images of the same CIFAR class (truck):



On the left, a synthetically generated image of a truck, on the right, a real photo of a truck

---

For preprocessing the data, Neural Networks benefit from some kind of Normalization of the data, since it helps the models converge faster [ref]. However, for the VGG pre-trained model, I needed to apply the Standarization that the documentation indicates, which gives us the mean and standard deviation to apply to the data [ref]

```
mean=[0.485, 0.456, 0.406]
std=[0.229, 0.224, 0.225]
```

Thus, the transformers for the data are the following:

```
transformer = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize(
                mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225],
            ),
        ]
    )
```

Network created

In my proposal I indicated I would fine-tune the model Inception_v3, conceived in the paper *Rethinking the Inception Architecture for Computer Vision* [ref]. However, after further investigation, I discovered that this network requires the input size to be 299x299 [ref]. CIFAKE has 32x32 images, which when resized as big as Inception_v3 requires, made this network quite slow to classify, and the resizing did not add value to the images.

I then investigated which pre-trained networks are available for 32x32 images, and found VGG as a good candidate [ref]. VGG19, the latest version available, is a convolutional network that has top-1 accuracy of 74%.

I added to VGG19 one final output layer which converts the number of output features of the original network to 1 final output neuron and a Sigmoid activation function, which will give a result between 0 and 1 for each image (this function is not defined explicitly in the network, since the loss function implementation used already applies it). The closer to 0, the more confident the network is that the image is fake, and the closer to 1, the more confident it's real.

The loss function used is Binary Cross Entropy Loss, which penalizes high confidence predictions on wrong classes, measuring the distance between the prediction and the correct label. BCELoss is dangerous when the dataset is unbalanced, but since we count with the same number of images for both classes, this is not an issue in our case [ref].

For the specific Pytorch version of this model, I will actually use the class BCEWithLogitsLoss, this loss function combines the Sigmoid activation function and the final Binary Cross Entropy Loss, making it more stable than applying them separately [ref].

Lastly, the optimizer chosen for this task is the ADAM optimizer, this optimizer is very robust and useful with deep, complex Neural Networks [ref]

Hyperparameter Tuning

The first step to create the model is to tune the Hyperparameters. For this I created a python script, `hpo.py` which contains the creation, training, and testing of the Neural Network. The model is trained for a maximum of 100 epochs, but the training stops when it detects that the Training loss has not decreased between epochs. This will accelerate the hyperparameter tuning, after which a full model with the chosen hyperparameters will be trained for all 100 epochs.

The tuner trains 5 different models to find the lowest Loss (defined as the Binary Cross Entropy) between multiple combinations of Learning Rate and Batch Size. This is performed in an instance type `ml.p3.2xlarge`. Due to the restrictions of the Udacity AWS account, only one of these jobs can be run at one time, since only one instance of this type is available. This instance has access to a GPU, so I configured the model to run in the GPU if available.

After the tuning, here is the final result for all five training jobs:

Rank	Learning rate	Batch Size	Epochs trained for	Loss on Testing dataset	Accuracy on Testing dataset	Recall on Testing dataset
#1	0.0040...	2048	4	0.535	73.45%	73.25%
#2	0.0039...	1024	3	0.540	73.27%	77%
#3	0.0035...	512	5	0.542	73.22%	78.7%
#4	0.0033...	256	4	0.602	70.07%	54.36%
#5	0.0025...	128	4	0.617	71.21%	66.07%

The chosen hyperparameters are:

- Learning Rate: 0.004035719481029806
- Batch Size: 2048

After 4 epochs, these parameters achieved an accuracy of 73.45% on the test dataset.

During the training, the hyperparameters tuned, and loss, accuracy and recall between epochs and every 100 batches is recorded in CloudWatch:

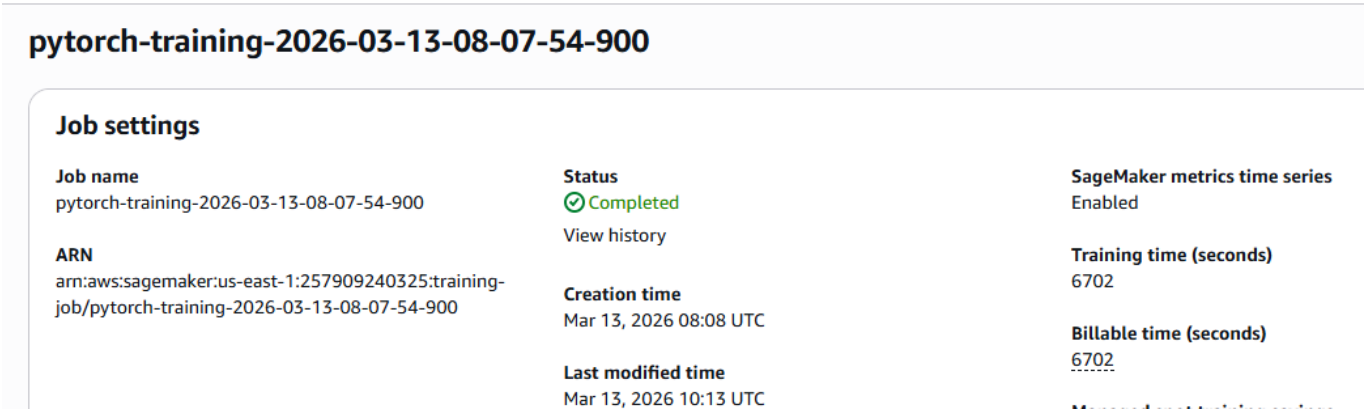
```
2026-03-12 13:14:03.063 | INFO | __main__:main:249 - Creating a model with the following parameters:
2026-03-12 13:14:03.063 | INFO | __main__:main:250 - Loss function: Binary Cross Entropy Loss
2026-03-12 13:14:03.063 | INFO | __main__:main:251 - Optimizer: ADAM with learning rate: 0.003320901466032771
2026-03-12 13:14:03.063 | INFO | __main__:main:252 - Batch Size: 256
2026-03-12 13:14:03.063 | INFO | __main__:main:253 - Train data directory: /opt/ml/input/data/train
2026-03-12 13:14:03.063 | INFO | __main__:main:254 - Test data directory: /opt/ml/input/data/test
2026-03-12 13:14:03.063 | INFO | __main__:main:255 - Outputs will be written to: /opt/ml/output/data
2026-03-12 13:14:03.063 | INFO | __main__:main:256 - Model will be saved to: /opt/ml/model
```

```
2026-03-12 13:16:58.581 | INFO     | __main__:train:151 - [Batch 300/391] Processed 77056/100000 (77.06%)
                          Average Loss achieved in this batch: 0.51
                          Current Total Loss: 46690.80

                          Correctly classified images in this batch: 196/256
                          True positives in this batch: 107/256
                          False negatives in this batch: 23/256

                          Current accuracy: 70.05%
                          Current recall: 70.41%
2026-03-12T…      2026-03-12 13:17:12.479 | INFO | __main__:train:170 - Epoch 2/100 finished. Loss: 0.6055, Accuracy: 0.7017, Recall: 0.7043
```

Estimator Training

After deciding on the best hyperparameters to use, I trained the model the full 100 epochs, setting up hooks for the SageMaker Profiler to check for issues during training, and the Debugger to check for multiple rules. This training was also done on an instance type `ml.p3.2xlarge`, I didn't utilize multi-instance training because, as previously stated, only one instance of this type is allowed at the same time in the Udacity AWS account.

After two hours, the training was complete, with a final Test accuracy of **74.92%**, and a Recall of **71.79%**

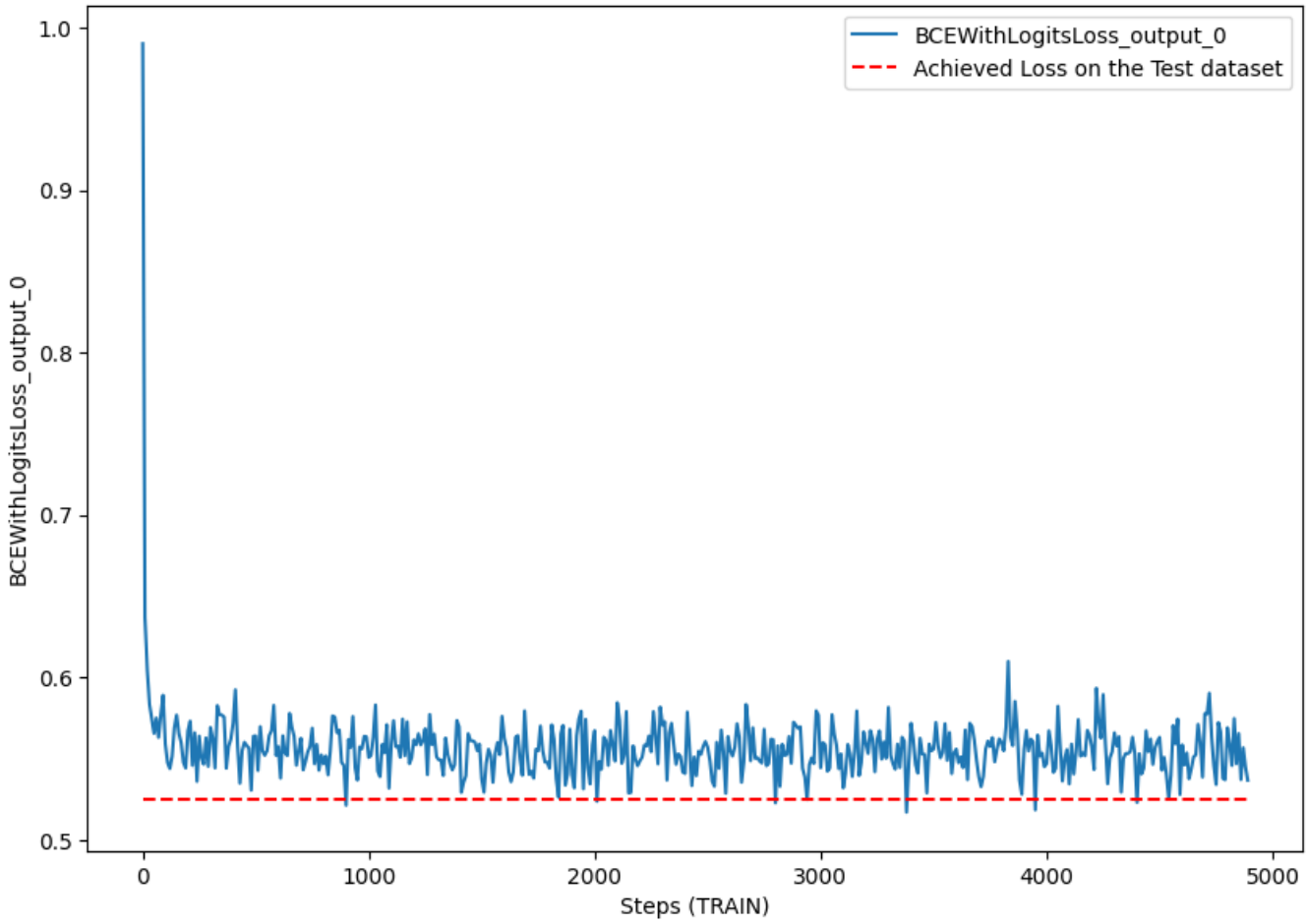


I activated the following Debugger rules:

- **Loss Not Decreasing**: detects when the loss function is not decreasing enough between training steps.
- **Vanishing gradient**: detects if the gradients become extremely small or drop to 0.
- **Exploding tensor**: detects if the tensors have infinite or nan values.

During training **NONE** of the rules triggered.

Here plotted is the loss function for the training dataset, the horizontal line denotes the final achieved loss evaluated in the test dataset:



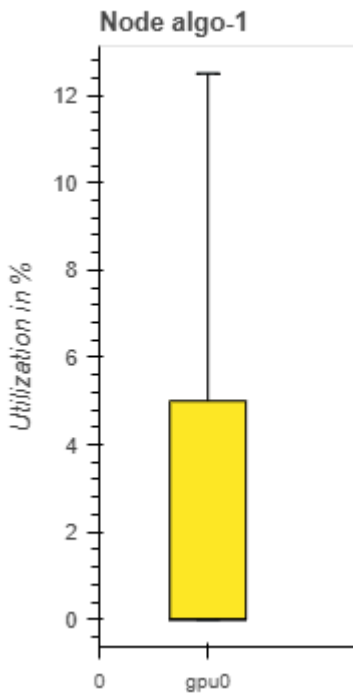
The plot shows a reduction in the loss function, however it's clear that it didn't improve much after the first few steps, which means that more iterations probably wouldn't have improved the network. If given more time, I would try a smaller learning rate value, since high values make the network get "stuck" with worse losses [ref].

The Profiler report can shed light on any issues with the utilization of resources. The training job triggered the following alerts:

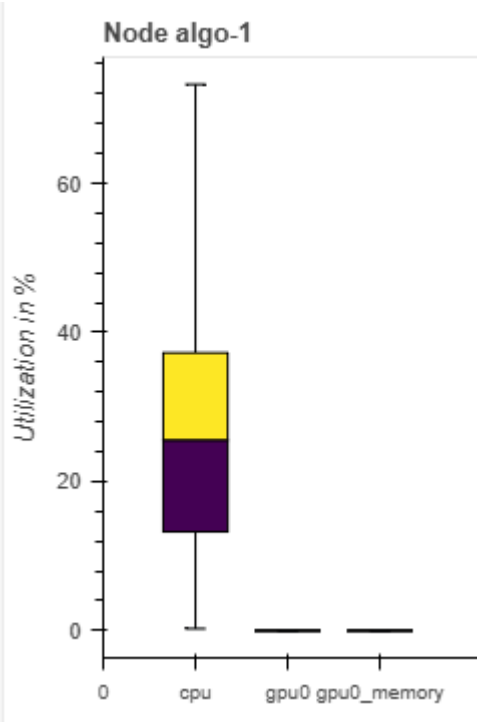
- **LowGPUUtilization**: Checks if the GPU utilization is low or fluctuating. This can happen due to bottlenecks, blocking calls for synchronizations, or a small batch size.
- **BatchSize**: Checks if GPUs are underutilized because the batch size is too small. To detect this problem, the rule analyzes the average GPU memory footprint, the CPU and the GPU utilization.

- **CPUBottleneck**: Checks if the CPU utilization is high and the GPU utilization is low. It might indicate CPU bottlenecks, where the GPUs are waiting for data to arrive from the CPUs. The rule evaluates the CPU and GPU utilization rates, and triggers the issue if the time spent on the CPU bottlenecks exceeds a threshold percent of the total training time. The default threshold is 50 percent.

The profiler detected that GPU utilization fluctuated a lot:

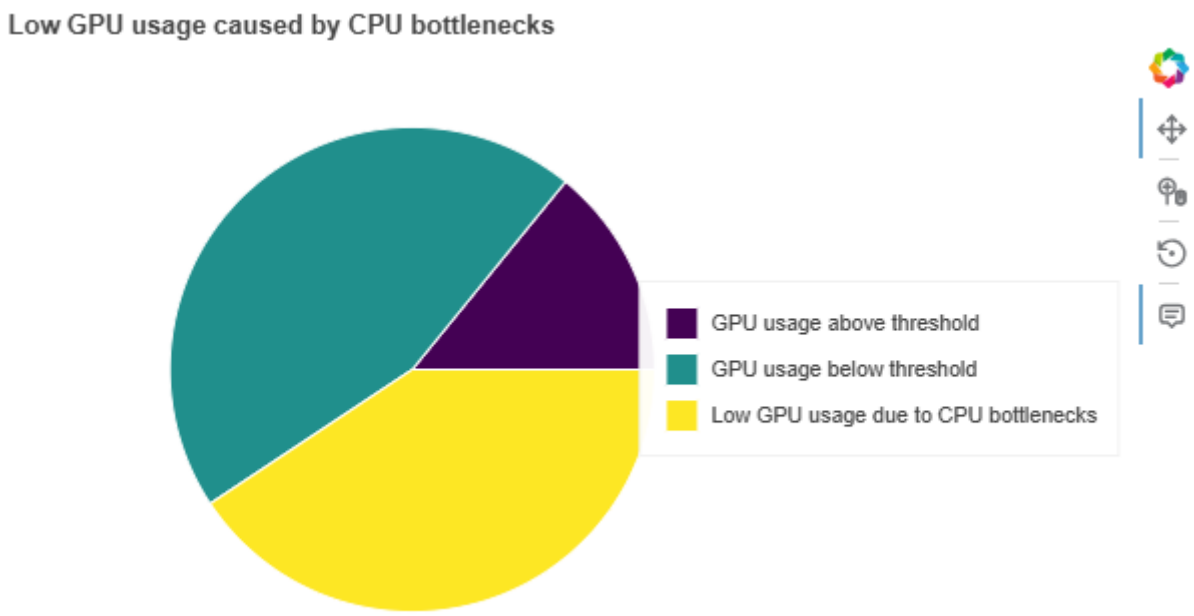


The BatchSize rule was triggered because it detected that the Batch Size was too small for the instance chosen. This means I could potentially save costs by lowering the instance, or make the training faster by increasing the batch size.



Lastly, the CPU bottleneck was triggered because CPU usage was very high and GPU usage low. The profiler detected that the CPU acted as bottleneck 68% of the time. It displays a plot that shows which percentage

of time the GPU usage was low due to the CPU:



In conclusion, the training of this model uses an instance which has a GPU but has only 8 CPUs, causing the instance to be infra-utilized. As a point of improvement, the model could be retrained using an instance that doesn't have a GPU, or with an instance with a higher number of CPUs, however this last measure would cause the price to increase.

The full Profiler report is available in the file `profiler-report.html`.

Model Deployment and Inference

The model is deployed from the trained estimator to an endpoint. The instance used is `ml.t2.medium`, which is the cheapest instance available for inference, and it's more than enough to attend the small amount of traffic in this project.

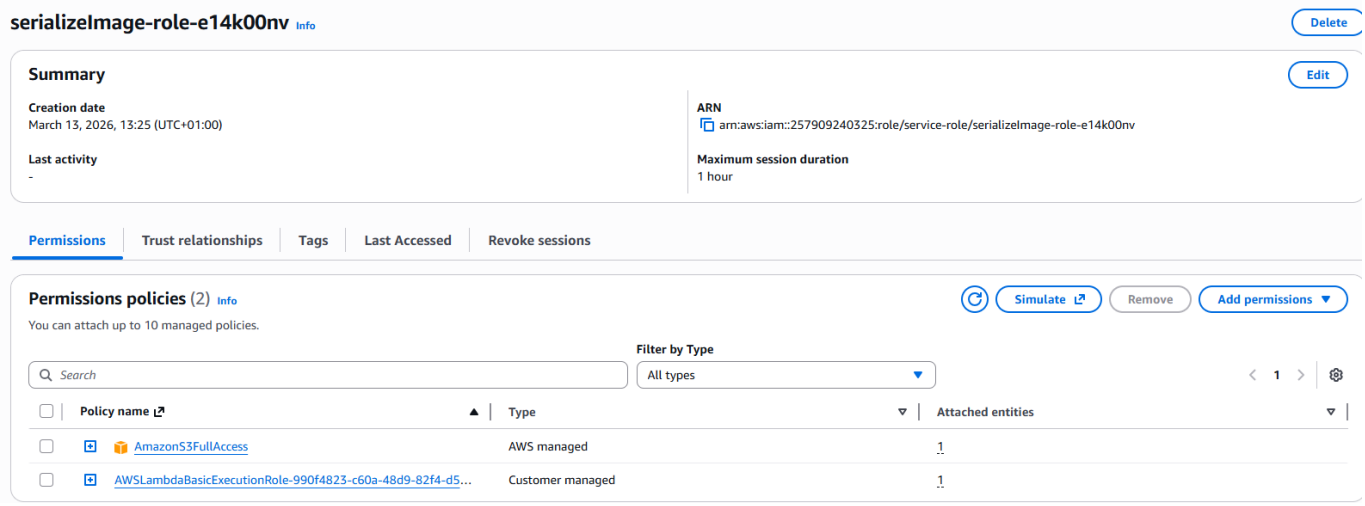
I then tested it with an example of a fake image from the test dataset, which the network correctly identified with a confidence of 69%.

Step Functions and Lambda for Inference

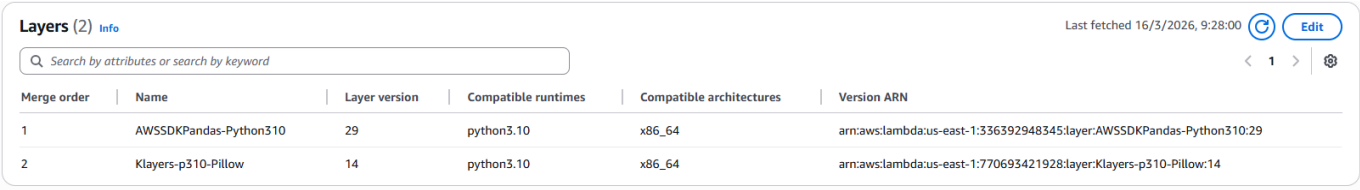
Finally, I created three lambda functions, which orquestrated with a Step Functions workflow, capture an image uploaded to the s3 bucket: `detecting-synthetic-images-sagemaker`, serialize it, asks the sagemaker endpoint for classification as real or fake, and checks against a threshold if the inference is confident enough to output to the user.

Lambda 1 - serializeImage

This lambda is created with a runtime of Python3.10 and assigned an execution role that has access every bucket in S3, to ensure it can access any image that is passed to it.



This lambda reads from the input the bucket and path to the image, saves it to a temporary folder, loads it as a `numpy` array and then serializes it using `pickle`. Besides numpy, this lambda needs the `pillow` library as dependency, we achieve this by adding two Lambda Layers, one with `pandas` and the other one with `pillow`:



The code is in the file `src/lambdas/serializeImage/lambda_function.py`.

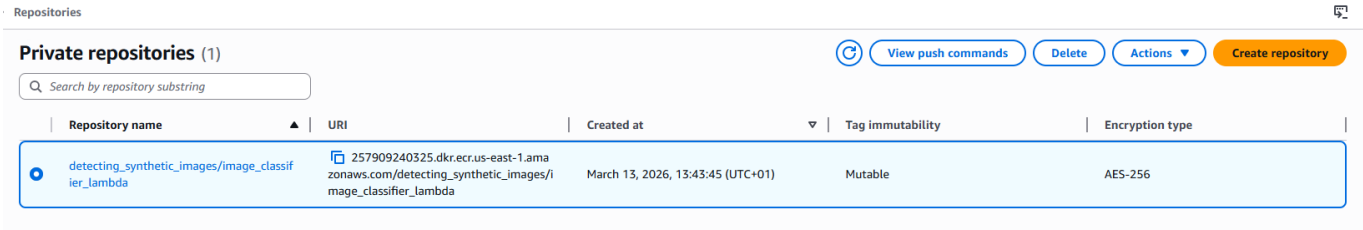
Lambda 2 - classifyImage

This lambda needs to access Torchvision to apply the transformers, which is not installed by default in Lambda. This can be achieved with layers, unfortunately, lambda layers have a size restriction that doesn't allow the zip files uploaded to define them to be too big.

Thus, this lambda is created as a **containerized** lambda. This kind of function is defined with a `Dockerfile`, `lambda_function.py` and a `requirements.txt` file, with which the torchvision library among others can be installed.

The files are under `src/lambdas/imageClassifier`.

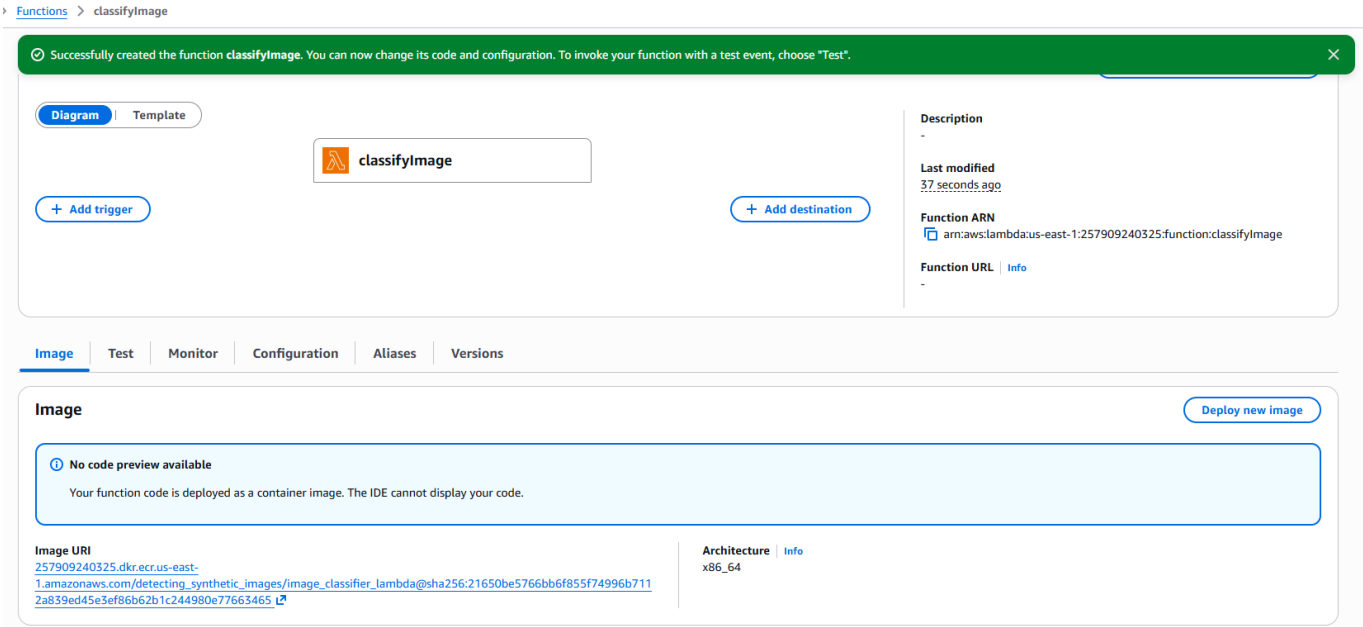
I first created a new private repository with AWS ECR (Elastic Container Registry):



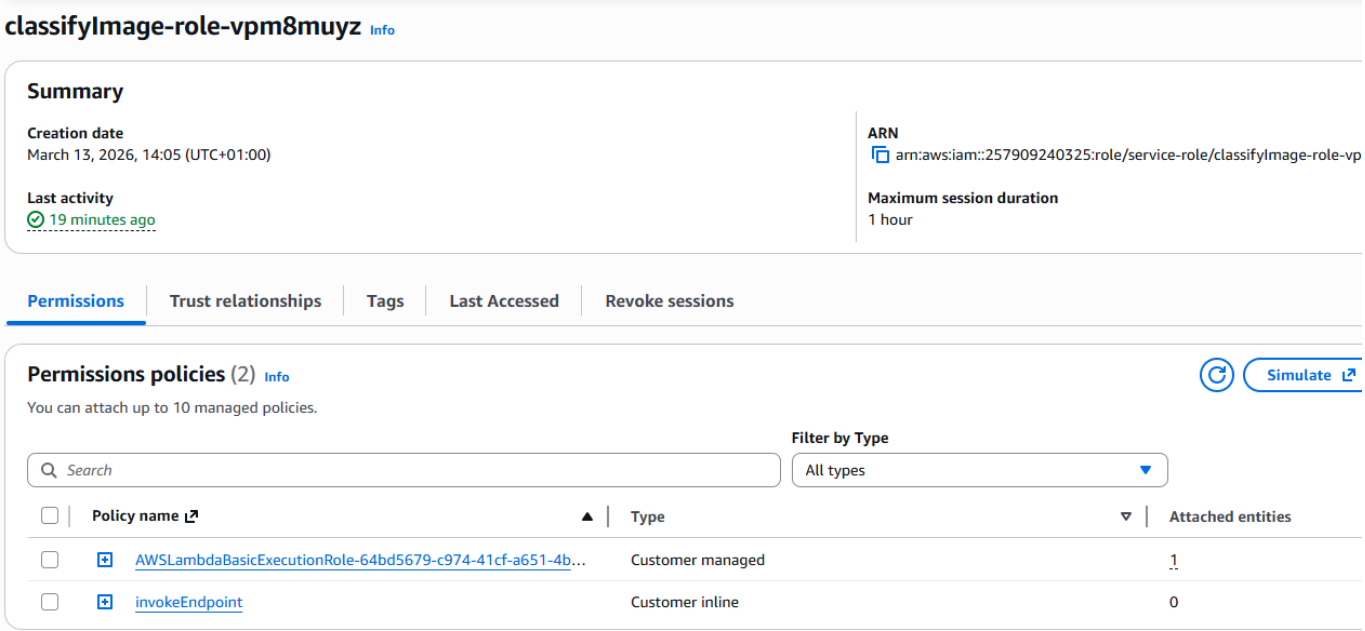
Then I followed the instructions to push the Docker image into the repository:

```
> docker build -t detecting_synthetic_images/image_classifier_lambda . --
provenance=false
> docker tag detecting_synthetic_images/image_classifier_lambda:latest
257909240325.dkr.ecr.us-east-
1.amazonaws.com/detecting_synthetic_images/image_classifier_lambda:latest
> docker push 257909240325.dkr.ecr.us-east-
1.amazonaws.com/detecting_synthetic_images/image_classifier_lambda:latest
```

After which I created the lambda function as a container function using the image:

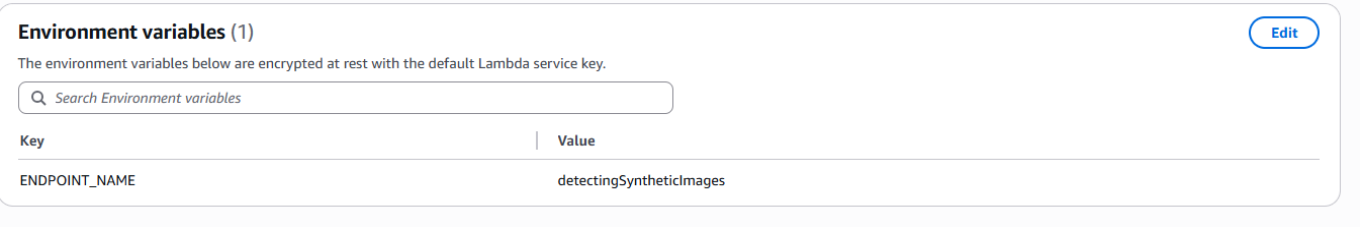


I attached an inline policy to the execution role `classifyImage-role-vpm8muyz` to invoke the Sagemaker Endpoint:

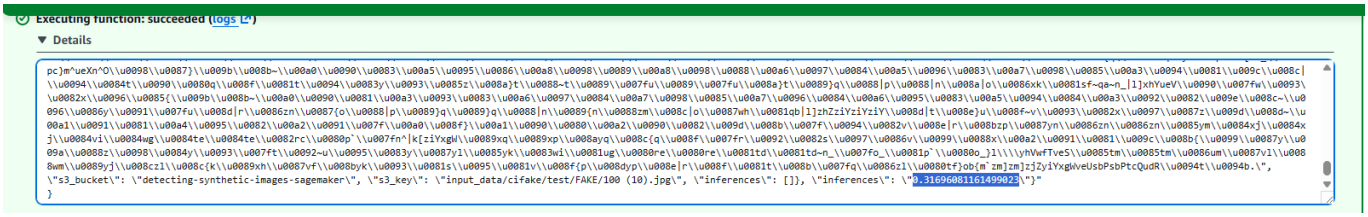


I also configured the Lambda's timeout to be 5 minutes, since the default of 3 seconds is not enough to transform the image and receive the inference.

Since the endpoint name can change, I defined an environment variable in the Lambda with the name of the endpoint, which the function uses.

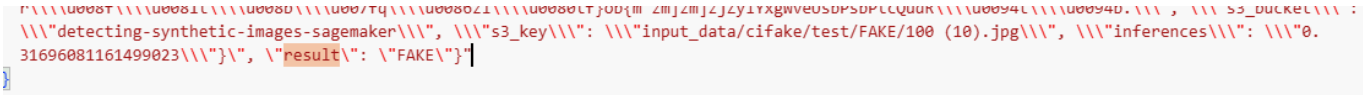


This function returns an atribute 'inference' with the float value for the prediction:



Lambda 3 - checkThreshold

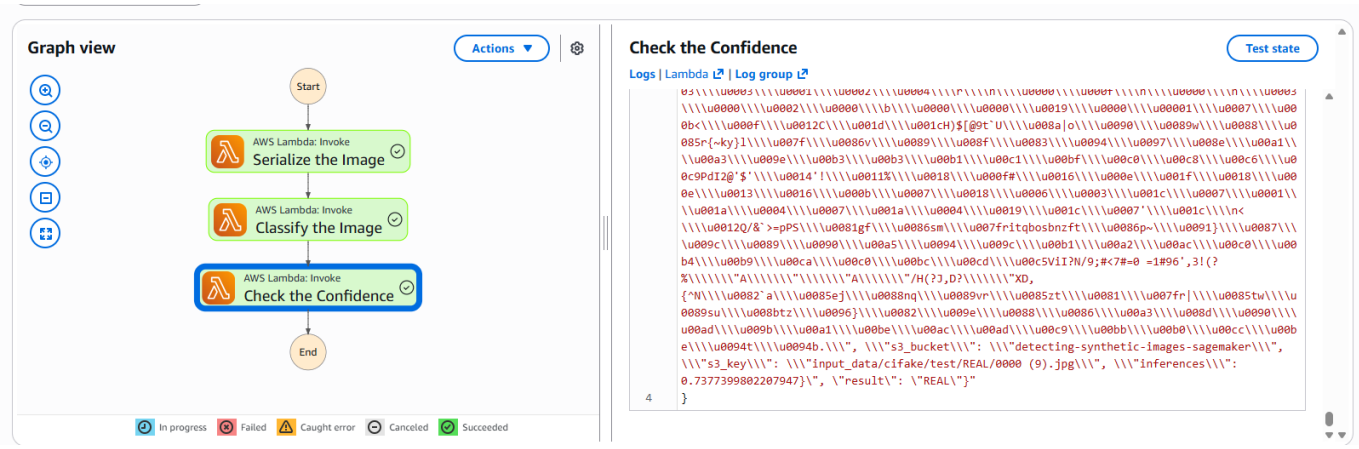
This last function takes the response of the previous one as input, and fails in case the prediction confidence is under a threshold (by default, 60%). In case that the prediction confidence is over the treshold, it returns an object with a 'result' attribute that determines whether the image is REAL or FAKE:



Step Functions

As the last step, I put together all three functions in a Step Functions workflow, called detectSyntheticImage. This workflow can be called with an s3_bucket and s3_key pointing to a JPG image, and it will output:

- 'REAL' as the `result` attribute of the response if it was detected with confidence that the image is real.
- 'FAKE' as the `result` attribute of the response if it was detected with confidence that the image is fake.
- The last function will **fail** if the network didn't return whether the image was REAL or FAKE with enough confidence.



Conclusion